QA Practice

Manual Testing Fundamentals — Notes & Practice

by Muhammad Danial Arshad

danialj751@gmail.com | +92 317 6125814

Contents

Part 1 — Software Development Life Cycle, Agile & Scrum

1. Introduction

Software testing is one of the most important activities in the entire process of building software, yet it is often the one that is least understood by people who are new to the field. In simple words, testing is the process of checking whether a software product actually does what it is supposed to do, and whether it does it correctly, safely, and in a way that satisfies the real needs of the end user. Before a tester can become effective at finding bugs, they first need to understand where testing fits into the bigger picture of how software is actually built, because testing that happens in isolation, without any connection to the rest of the development process, rarely adds real value to a project.

This section focuses on two closely related ideas that form the foundation of a good quality assurance mindset. The first is the Software Development Life Cycle, commonly known as the SDLC, which describes the stages that a software product passes through from the moment it is only an idea until it is delivered to users and maintained afterward. The second is understanding how modern teams, especially those working in Agile and Scrum environments, organise their work, and where exactly Quality Assurance, or QA, fits into that organisation.

2. Understanding the Software Development Life Cycle

The Software Development Life Cycle is basically a roadmap that development teams follow to turn a business idea into a working software product. Almost every SDLC model includes some version of the following steps: requirement gathering and analysis, planning, design, development or coding, testing, deployment, and maintenance. The cycle usually begins with requirement gathering, where business analysts, product owners, and sometimes clients sit together to figure out exactly what the software needs to do. Once requirements are reasonably clear, the team moves into planning and design, where decisions are made about the technical architecture of the system and how the different parts of the software will interact with each other.

After the design stage, developers begin writing the actual code that brings the design to life. Once a reasonable portion of the code is ready, it moves into the testing stage, where testers examine the software to check whether it behaves as expected and is free of defects that could cause it to fail in the hands of real users. After testing is complete, it is deployed, and the maintenance stage covers everything that happens after release, including fixing bugs discovered later and adding new features.

It is worth noting that the SDLC is not always followed in a strict, one-directional order. Older approaches such as the Waterfall model treat these stages as a straight line, where one stage must be completely finished before the next begins, and going back to an earlier stage is difficult and expensive. Modern software teams rarely work this way anymore, since the Waterfall approach makes it very hard to respond to changing requirements or unexpected problems discovered late in the project, which is one of the main reasons Agile approaches became so popular.

3. The Agile Approach to Software Development

Agile is best understood not as a single fixed method but as a set of values and principles for organising software work in a flexible and collaborative way. Instead of trying to plan an entire project in complete detail before any coding begins, Agile teams break their work into small, manageable pieces and build the product gradually, gathering feedback along the way and adjusting their plans as they learn more. This means an Agile team essentially runs many small SDLC cycles back to back, each one producing a small but usable increase in the functionality of the product.

One of the biggest advantages of the Agile approach is that it allows problems to be discovered and corrected much earlier than in a traditional Waterfall project. Because the team is constantly building, testing, and reviewing small pieces of the product, mistakes in requirements, design, or code tend to surface quickly rather than staying hidden until the very end of the project, when they would be far more expensive and difficult to fix.

4. Scrum as a Practical Framework for Agile Teams

Scrum is one of the most widely used frameworks for actually putting Agile values into practice. A Scrum team typically organises its work into fixed time periods called sprints, usually lasting between one and four weeks. At the start of a sprint, the team holds a sprint planning meeting to decide which items from the product backlog will be tackled during that sprint. Throughout the sprint, the team holds short daily standups, where each member briefly shares what they worked on, what they plan to work on next, and any blockers.

At the end of each sprint, the team holds a sprint review, where completed work is demonstrated to stakeholders, followed by a sprint retrospective, where the team reflects on what went well and what could be improved. A Scrum team is generally made up of three roles: the Product Owner, who decides what should be built and in what priority; the Scrum Master, who helps the team follow the Scrum process and removes obstacles; and the Development Team, who collectively turn backlog items into finished, working software during each sprint.

5. Where QA Fits into Agile and Scrum Teams

In older, Waterfall-style projects, testing was traditionally treated as a separate phase that only started once developers had finished writing all the code. In Agile and Scrum environments, this has changed considerably. QA is no longer a final checkpoint bolted onto the end of development; instead, it becomes an ongoing activity that runs alongside coding throughout every sprint. This shift is often described using the phrase shift-left testing, meaning testing activities move earlier in the development timeline.

In a Scrum team, testers are usually full members of the Development Team. They take part in sprint planning to help clarify requirements, write test cases as soon as new functionality becomes available, perform manual testing on features still being built, and report defects quickly so developers can fix them within the same sprint. Testers also attend daily standups and contribute to sprint reviews and retrospectives, protecting product quality at every stage of the sprint cycle.

6. The QA Mindset

Becoming a good tester also requires developing a particular way of thinking often called the QA mindset, built around curiosity, scepticism, and attention to detail. While a developer's main goal is usually to make a feature

work, a tester's mindset is oriented toward asking how a feature might fail, what unusual inputs a user might provide, and what edge cases might have been overlooked. A strong QA mindset also involves thinking from the perspective of the end user, constantly asking whether a feature is not only functionally correct but also genuinely usable and free of confusing behaviour.

7. Conclusion

The Software Development Life Cycle provides the overall structure within which software is planned, built, tested, released, and maintained, while Agile and Scrum represent modern ways of moving through that cycle in small, flexible steps. Within this environment, Quality Assurance is an ongoing, embedded responsibility that runs through every sprint. Developing a genuine QA mindset, built on curiosity and a constant focus on the end user, is just as important as learning specific testing techniques.

Part 2 — Software Testing Principles

1. Introduction

Software testing principles are the basic guidelines that experienced testers rely on to think about quality in a structured and consistent way. They are lessons that the testing community has learned over many decades of building and breaking software. A tester who understands these principles is able to plan testing work more intelligently, set realistic expectations with stakeholders, and avoid common traps that lead teams to believe their software is safer or more reliable than it actually is. This section looks at four of the most fundamental testing principles: early testing, defect clustering, the pesticide paradox, and the absence of errors fallacy.

2. Early Testing

The principle of early testing states that testing activities should begin as early as possible in the software development life cycle, ideally starting from the requirements and design stages rather than waiting until coding is finished, an idea often summarised as shift-left testing. The earlier a defect is found, the cheaper and easier it is to fix. A mistake in a requirement document might take only a few minutes to correct if caught while the document is still being reviewed, but if it is not discovered until after the feature has been fully coded and released, fixing it costs far more time and money.

In practice, early testing means testers review requirement documents for gaps and ambiguities, and start thinking about test cases even before a single line of code has been written. In Agile teams, this often shows up as testers participating in sprint planning, asking clarifying questions, and preparing test scenarios in parallel with developers.

3. Defect Clustering

Defect clustering refers to the well-documented tendency for a small number of modules or components in a software system to contain a disproportionately large share of the total defects found during testing, related to the Pareto principle where roughly eighty percent of problems come from around twenty percent of causes. Certain modules, perhaps unusually complex ones or those modified frequently by many developers, end up far buggier than the rest of the application.

Understanding defect clustering is useful for planning testing efficiently. Experienced testers pay close attention to which parts of the application have historically produced the most defects, directing extra attention and more thorough regression testing toward those high-risk areas, rather than treating every module as equally risky.

4. The Pesticide Paradox

The pesticide paradox takes its name from an analogy with farming: if the same pesticide is used repeatedly, surviving pests develop resistance, so the pesticide gradually becomes less effective. In software testing, if a tester keeps running exactly the same set of test cases over and over without ever updating them, those tests will gradually stop finding new bugs, not because the software has become perfect, but because all the defects that set of tests was capable of catching have already been found.

The practical lesson is that test cases and test suites need to be reviewed, updated, and expanded regularly rather than treated as a fixed checklist. As a product evolves, testers need to keep asking new questions, exploring different input combinations, and sometimes deliberately trying testing techniques they have not used before.

5. Absence of Errors Fallacy

The absence of errors fallacy points out a mistaken belief: that if testing has found very few or zero defects, the software must automatically be good and ready for release. In reality, a low bug count does not necessarily mean high quality; it might simply mean testing has not been thorough enough, or the tests were not aligned with what users actually need. A system could be almost completely free of technical bugs and still fail commercially because it does not solve the right problem.

This principle reminds teams that testing is not only about confirming the absence of defects, but about verifying that the software genuinely meets user needs. Testing teams are encouraged to combine verification, checking that software was built correctly according to its specification, with validation, checking that it was built to solve the right problem in the first place.

6. How These Principles Work Together

Early testing helps a team catch problems before they become deeply embedded in the codebase, which naturally reduces defect clustering. Awareness of defect clustering helps testers decide where to concentrate limited effort, but that effort remains useful over time only if the pesticide paradox is also kept in mind. Finally, the absence of errors fallacy reminds the whole team that finding few bugs is not the ultimate goal of testing; delivering software that genuinely satisfies real user needs is.

7. Conclusion

Together, early testing, defect clustering, the pesticide paradox, and the absence of errors fallacy form a set of practical lessons that help testers think critically about their own work. A tester who internalises these four principles is far better equipped to make sound decisions about how to plan, prioritise, and continuously improve their testing work throughout a project.

Part 3 — Types of Software Testing

1. Introduction

A ride-hailing app example aside, a QA tester needs to understand that not all testing looks the same or serves the same purpose. Testing activities are usually grouped into a few broad categories depending on what aspect of the software is being examined. This section looks at four major categories: functional testing, non-functional testing, structural testing, and change-related testing. Each answers a different question about the software, and a mature testing strategy usually draws on all four rather than relying on just one.

2. Functional Testing

Functional testing is concerned with checking whether the software does what it is supposed to do, based directly on its documented requirements. It focuses on the visible behaviour of the software from the perspective of an end user. A tester performing functional testing on a login form would check whether entering correct credentials logs the user in, whether an incorrect password produces an appropriate error, and whether missing required fields is handled sensibly.

Because functional testing is grounded in requirements, it depends heavily on those requirements being reasonably clear and complete. Functional testing techniques commonly include verifying individual features, checking how features interact, and confirming business rules are applied correctly.

3. Non-Functional Testing

While functional testing asks whether software does the right thing, non-functional testing asks how well it does it. This covers qualities such as performance, usability, security, reliability, and scalability. A system might pass every functional test perfectly and still be a poor product if it takes ten seconds to respond, crashes under load, or exposes sensitive data.

Performance testing examines how quickly the system responds under load. Usability testing looks at how easy and pleasant the software is to use. Security testing examines whether data is properly protected. Reliability testing looks at whether the system behaves correctly over long periods of continuous use.

4. Structural Testing

Structural testing, sometimes called white-box testing, looks directly at the internal structure of the code rather than only its external behaviour. It asks whether every line of code, every decision branch, and every logical path has actually been executed and verified during testing. This type of testing is typically performed by developers or testers with programming skills.

Structural testing is often measured using coverage metrics. Statement coverage measures what percentage of individual lines have been executed at least once, while branch coverage measures whether every decision point has been tested in both directions. Low coverage numbers are a strong warning sign that significant portions of the application have never actually been tested.

5. Change-Related Testing

Change-related testing focuses on what happens to software quality whenever something is modified, whether a small bug fix or a new feature. Changing one part of a system can unintentionally affect other parts that seemed unrelated, and the only way to catch these unintended side effects reliably is to test the system again after every meaningful change.

The most common form is regression testing, re-running previously passing test cases after a change to confirm the change has not broken existing functionality. As projects grow larger and change more frequently, especially in Agile environments, regression testing becomes increasingly important, and many teams automate their regression suites so this checking can happen quickly and consistently.

6. How These Categories Complement Each Other

A genuinely thorough testing strategy draws on all four categories rather than treating any single one as sufficient. Functional testing confirms a feature behaves correctly, non-functional testing confirms it performs well under realistic conditions, structural testing confirms the underlying code has been thoroughly exercised, and change-related testing confirms quality is preserved every time the system evolves. Skipping any one category tends to leave a corresponding blind spot.

7. Conclusion

Functional testing checks that software does the right thing, non-functional testing checks that it does that thing well, structural testing checks that the underlying code has genuinely been verified, and change-related testing checks that quality is preserved as the system evolves. An experienced tester learns to combine them thoughtfully based on a project's risks and available time.

Part 4 — Black Box, White Box & Grey Box Testing

1. Introduction

When testers talk about how much access they have to the internal code of an application while testing it, they describe their approach using one of three terms: black box testing, white box testing, or grey box testing. These labels describe how much visibility a tester has into the internal workings of the software, not separate categories of bugs or project stages. Understanding the differences helps a tester choose the right technique for a given situation.

2. Black Box Testing

Black box testing is an approach in which the tester evaluates the software purely from the outside, focusing entirely on inputs and outputs without any knowledge of the internal code or logic. A tester using this approach only needs a clear description of what the software is supposed to do, making it well suited to testers without a programming background and to situations that closely mirror how an ordinary end user interacts with the product.

For example, testing a search bar on a shopping website by typing various search terms and observing the results is black box testing. If a relevant product does not appear or the site crashes on an unusual character, that is a defect worth reporting, regardless of what is happening in the underlying search algorithm.

3. White Box Testing

White box testing takes the opposite approach, giving the tester full visibility into the internal structure of the code, including its logic, data flow, and every path the code can take. It is typically performed by developers or testers comfortable working directly with code, with the goal of verifying the internal implementation is correct and every meaningful path has been exercised.

As an example, a tester using white box testing on a shipping discount function would look directly at the code and deliberately test values exactly at the discount threshold, just above it, and just below it, since these boundary conditions are where off-by-one mistakes commonly occur.

4. Grey Box Testing

Grey box testing sits between these two extremes, combining benefits of both. The tester has partial knowledge of internal workings, such as the general architecture or database structure, without complete access to every line of source code. This partial knowledge allows smarter, more targeted test cases while still largely testing from the outside.

A common example involves testing a web form while knowing the underlying database field has a maximum length, allowing the tester to deliberately check what happens when an unusually long value is entered, even without reading the code that processes the form.

5. Choosing the Right Approach

In real projects, teams rarely rely on only one approach. Black box testing validates that software genuinely meets user expectations without requiring deep programming knowledge. White box testing is essential for catching subtle logical errors but demands solid technical skills and code access. Grey box testing offers a practical middle ground, especially useful for testing integrations between systems.

6. Conclusion

Black box, white box, and grey box testing describe three different levels of visibility into a system's internal workings. Black box focuses on validating behaviour from a user's perspective, white box focuses on verifying the underlying code is logically sound, and grey box combines partial internal knowledge with an external testing perspective. Recognising which approach is being used allows a tester to plan more deliberately and communicate clearly about exactly how software has been verified.

Part 5 — Test Case Writing

1. Introduction

Writing a good test case is one of the most fundamental skills a QA tester needs to develop, since a test case turns a general idea like "check that login works" into something precise, repeatable, and unambiguous enough that any tester can execute it and get the same result. This section walks through the standard components of a professional test case: the test ID, the title, the preconditions, the test steps, the expected result, and the actual result.

2. Test ID

The test ID is a unique identifier assigned to every test case so it can be referenced unambiguously in reports and defect tracking tools, without repeating the entire title every time. A typical test ID follows a short, consistent naming convention, such as TC_LOGIN_001. Consistency matters more than the exact format, since it lets anyone on the team quickly tell which feature area a test case belongs to.

3. Title

The title is a short, descriptive summary of what a test case is checking, written so anyone scanning a list of test cases can immediately understand its purpose. A good title is specific: "Verify login with valid username and password" tells far more than a vague "Login test." Because a single feature is usually covered by many test cases, titles need to clearly distinguish one from another.

4. Preconditions

Preconditions describe the state the system, environment, or test data needs to be in before the test steps can be executed. Test cases assume certain conditions are already true, and the preconditions section makes those assumptions explicit rather than leaving the tester to guess. Clearly stating preconditions prevents wasted time investigating what looks like a failure when the test simply could not be executed correctly.

5. Test Steps

The test steps list, in clear and specific order, the exact actions the tester needs to perform. Each step should describe a single, concrete action, written clearly enough that a different tester could follow it and arrive at the same point. Vague steps like "log in to the app" force every tester to make their own decisions, defeating the purpose of a repeatable test case.

6. Expected Result

The expected result describes, in specific and observable terms, exactly what should happen if the feature is working correctly. Without a clearly defined expected result, there is no objective way to judge whether a test passed or failed. A strong expected result avoids vague language and instead describes a concrete, checkable outcome.

7. Actual Result

The actual result records what the system genuinely did when the test steps were carried out, filled in only during test execution. If it matches the expected result, the test case passes; if it differs, it fails, and that discrepancy becomes the starting point for a defect report. Recording the actual result precisely, ideally with a screenshot, gives developers a faster starting point than a bare pass or fail label.

8. A Complete Test Case Example

Bringing all these components together, a complete test case for a login scenario looks like the example below.

Test ID	TC_LOGIN_001
Title	Verify login with valid username and password
Preconditions	A registered user account exists with a known, valid email address and password.
Test Steps	1. Navigate to the login page. 2. Enter a valid email. 3. Enter the correct password. 4. Click Log In.
Expected Result	The user is redirected to their dashboard within a few seconds, and a welcome message is displayed.
Actual Result	Matched the expected result. Status: Passed.

Every component plays a distinct role: the test ID makes the test easy to reference, the title makes its purpose clear at a glance, the preconditions make its assumptions explicit, the steps make it repeatable, the expected result makes it judgeable, and the actual result is where the judgement is recorded.

9. Test Planning

While a test case describes how to verify one specific scenario, test planning operates at a higher level, covering decisions about what needs to be tested overall, in what order, and with what resources, before individual test cases are even written. A test plan identifies scope, testing types, and available resources, giving structure and direction to the process of writing individual test cases.

10. Conclusion

A well-written test case is built from six components that each serve a distinct purpose: the test ID for reference, the title for quick understanding, the preconditions for explicit assumptions, the steps for repeatability, the expected result for objective judgement, and the actual result for recording what happened. Combining strong test case writing with sound test planning gives a QA team both direction and precision.

Part 6 — Test Suite Organization

1. Introduction

Once a tester understands how to write an individual test case, the next skill is thinking about coverage: making sure a test suite examines a feature from every angle that matters. This section explains four categories that form the backbone of a well-organized test suite: positive testing, negative testing, edge case testing, and boundary testing.

2. Positive Testing

Positive testing, or happy path testing, verifies that a feature behaves correctly when given valid, expected input used the way it was designed to be used. These test cases confirm the feature actually does its job. A thorough positive test suite covers more than one scenario, such as different valid discount codes and different letter cases if the system is meant to be case-insensitive.

3. Negative Testing

Negative testing verifies that a feature behaves safely and predictably when given invalid input. It should reject bad input gracefully with a clear error message rather than crashing. Negative testing is often undervalued by newer testers, but in real-world use, people very frequently do things that were not planned for, making this kind of testing critical.

4. Edge Case Testing

Edge case testing explores unusual, extreme, or rare scenarios that fall outside typical usage but are still technically possible. Edge cases are often where the most interesting bugs hide, precisely because the original developers may not have thought carefully about them. Good edge case testing requires creative "what if" thinking beyond the feature's obvious, everyday use.

5. Boundary Testing

Boundary testing is a more focused form of edge case thinking that specifically targets the exact limits of a valid input range, since defects are statistically far more likely to occur right at a boundary than in the middle of a valid range. If a field accepts 1 to 10, boundary testing checks 1, 10, and the values just outside, 0 and 11. Boundary testing is precise and predictable once the valid range is known, while broader edge case thinking requires more open-ended creativity.

6. Organizing a Balanced Test Suite

A well-organized test suite deliberately includes test cases from all four categories for every important feature. A common approach is to walk through a feature one category at a time: write positive cases first, then negative cases, then think through edge case scenarios, and finally identify numeric or length limits for boundary cases. Grouping test cases by category makes gaps easy to spot; a suite with ten positive cases but no negative cases is immediately recognisable as unbalanced.

7. Conclusion

Positive, negative, edge case, and boundary testing each answer a different question about a feature. Learning to organize test coverage deliberately around these four categories, rather than writing test cases in an unstructured way, is one of the most effective habits a QA tester can develop.

Part 7 — Test Estimation Techniques

1. Introduction

Test estimation is the process of predicting the effort, time, and resources needed to complete a testing activity. A team that cannot reasonably predict how long testing will take will either rush critical testing at the last minute or repeatedly miss release dates. This section explains three techniques commonly used together: work breakdown, three-point estimation, and historical data.

2. Work Breakdown

Work breakdown is the practice of dividing a large, difficult-to-estimate testing effort into smaller, more manageable pieces that are each individually easier to estimate accurately. Instead of guessing how long it takes to "test the whole application," a tester lists every module or activity, such as user login, checkout, and payment, each broken down further into writing test cases, executing them, and retesting fixed defects.

Work breakdown reduces the risk of forgetting something entirely. When a tester lists every module and category of work, it becomes far less likely an entire area or category, like regression testing, gets left out of the estimate altogether.

3. Three-Point Estimation

Three-point estimation asks for three separate estimates for each piece of work: an optimistic estimate (best case), a pessimistic estimate (realistic worst case), and a most likely estimate. These are combined using the PERT formula: Estimate = (Optimistic + 4 × Most Likely + Pessimistic) / 6. For example, with an optimistic of 2 days, most likely of 4, and pessimistic of 8, the estimate works out to (2 + 16 + 8) / 6 ≈ 4.3 days.

This formula gives the most likely estimate four times as much influence as either extreme, reflecting that it is the most probable outcome, while still letting the optimistic and pessimistic numbers pull the final estimate slightly. This is especially useful for tasks involving real uncertainty, such as testing an unfamiliar third-party integration.

4. Historical Data

Historical data means using records of how long similar testing work has actually taken on previous projects to inform new estimates, rather than estimating purely from intuition. If a team has recorded that testing a medium-complexity login feature consistently took three to five days across past projects, that history is valuable the next time a similar feature needs estimating.

The main advantage is that it corrects for a common human bias: people tend to be naturally overoptimistic when estimating from imagination rather than evidence. Historical data works best when applied to genuinely comparable work; a simple internal tool's login and a banking app's login with multi-factor authentication are not directly comparable despite having similar names.

5. Combining the Three Techniques

In real projects, these three techniques are almost always used together. A tester starts with work breakdown, then applies three-point estimation to each individual task, using historical data wherever available to ground the optimistic, most likely, and pessimistic numbers in real past experience rather than pure imagination. This combined approach produces estimates that are far more defensible than a single number pulled from intuition alone.

6. Conclusion

Work breakdown prevents scope from being forgotten, three-point estimation captures genuine uncertainty honestly, and historical data grounds estimates in real past experience. A tester who combines all three produces testing estimates that stakeholders can actually trust, and estimates that get more accurate over time as more historical data is collected.